machine, in our example, could likely just run a “coin-recognition” software, without keeping a count[6], and send, to the machines at level 2, lists containing unsorted occurrences of coins, something like:

```
$100: 1
50¢: 1
$100: 1
$1: 1
25¢: 1
…
```

Here, the info extracted by mappers is just the presence of a coin.

Then the machines at level 2, instead, would be specialized in counting. Every machine at level 2 would receive *all* the entries for occurrences of a certain group of coins, and do something with them (counting them, for example, but it could also sum up their values or filter them). This step is therefore called *Reduce*, because it takes info limited to a homogeneous group of entries, and combines (aka *reduces*) them to get our final result.

As mentioned, the key disadvantage of the classic, “flat” parallel computation is that composing the results of all the parallel threads/processes/machines would be the bottleneck of the whole process: if a single process has to spawn the threads and then get their results and combine them, it will still have to sequentially access the whole dataset at least once, and even if it also parallelizes the process that combines the intermediate results, it still remains a